Facebook app update
Facebook released a new update for its native Android app which adds faster photo viewing experience and voice messaging features

Filabot
A Kickstarted device called "Filabot" can turn household and printed plastic stuff into printable filaments for use in 3D printing

DIY: Make Your Own CMS

Sometimes it is more of an effort to adapt an existing CMS to your needs than it is to build one from scratch

As many wise people will tell you, avoid making your own CMS, make modules for an existing open source CMS instead. The are many reasons for this – more work, less people who understand your code, poor security. Yet, with the right framework all this can be avoided.

Content Management Systems usually revolve around the kind of content being managed, blog posts / articles, audio, video, images or something else entirely. If the data has a non-standard hierarchy then a traditional CMS system might not do.

Imagine a CMS for car parts that has a heirarchy of part, which could fit in any number of car, and which could work with any number of other parts. It is unlikely you will find something exactly for this.

So how about we make our own? In this DIY we will use Django, a popular web framework written in Python. In this example, we will make a movie database application. Here we have a movie, and each movie has people attached to it in different roles, director, producer, actor and so on.

If you are familiar with coding you should be able to follow this DIY quite easily.

Pre-requisites

To make this application, you need to have Python and Django installed. Rather than go through that here, we can point you to a good tutorial on the Django website: https://docs.djangoproject.com/en/dev/intro/install/

Next we need to set up our project. Create an empty directory for your projects files, and in it run the following command:

```
django-admin.py startproject mycms
```

We just created the basic structure of a Django project, a project in turn can include one or more "apps". We create that as follows:

```
python manage.py startapp movies
```

We created a single app, called "movies". An app provides a distinct bit of functionality. In a CMS, one app might manage articles, another the forum and a third could handle images and media. Apps can reference one another, so you could embed your media into a forum post or article.

The Django Project Structure

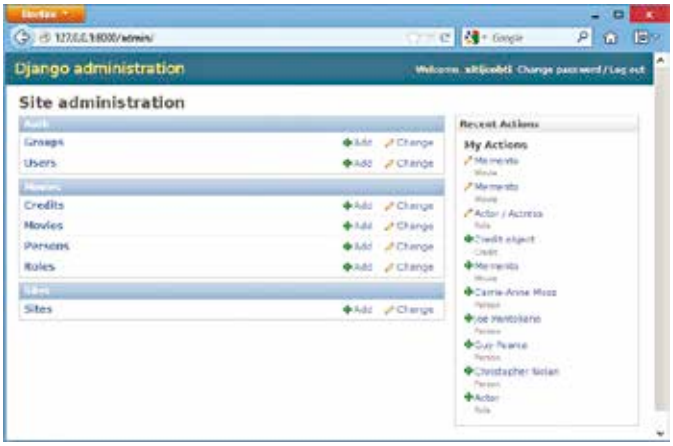
A Django project is a collection of apps configured to work together. You can write these apps, or find ready-made apps and integrate them into your own project.

At this point we have the basic structure of our project, so let's see what role some of the important files play:

1. **manage.py:** You use this to run commands on your project.
2. **mycms** folder: This folder has files related to your Django project.
3. mycms/**settings.py**: This is the file that has all the settings for the project.
4. mycms/**urls.py:** This file has a table of contents that maps URLs to features of our web app.
5. **movies** folder: This folder has files related to your movies app.
6. movies/**models.py**: Here we define the data structure of our website.
7. movies/**views.py**: This file has the code that actually returns HTML code to a user browsing the website.

Basic setup

We need to change some very basic settings before this project will work the way we want. First let's set up the database; open `settings.py` and look for the code beginning with `DATABASES`, it should be near the top.



A ready-made admin panel!

Here, change the `ENGINE` parameter to `django.db.backends.sqlite3` and set `NAME` to `test.db`. SQLite is a simple SQL engine that keeps everything in a file so we won't need to set up a server or anything.

Further down, in the `INSTALLED_APPS` code section, uncomment the line's first commented entry `'django.contrib.admin'` by removing the `#` and the space after it; remember, indentation matters a lot in Python.

We will also add our own app to the list of installed apps. Add the following line of code at the end inside the `INSTALLED_APPS` block:

```
'movies',
```

Like the previous step, mind the indentation.

Finally, open the `urls.py` file in the `mycms` folder and uncomment the lines for the "admin" as instructed by the comments on that file.